

CampusConnect

Campus Social Network — Requirements, Architecture & Proof of Concept

Assignment 1 Report — Omar Ibrahim, Salama Aldhaheeri, Makar Ulesov, Aro Dana

AI1220 — MBZUAI — October 2026

Repository: github.com/omar07ibrahim/campus-social-network

Contents

1. [Product Scope](#)
2. [1.1 Stakeholders](#)
3. [1.2 Functional Requirements](#)
4. [1.3 Non-Functional Requirements and 2.1 Architectural Drivers](#)
5. [1.4 User Stories and Scenarios](#)
6. [1.5 Validation and Traceability](#)
7. [2.2 C4 System Context](#)
8. [2.2 C4 Containers](#)
9. [2.2 C4 Components \(Backend API\)](#)
10. [2.3 Behaviour and Design Decisions](#)
11. [2.3 Collaborative Sequence Diagram](#)
12. [2.4 Interfaces](#)
13. [2.5 Code and Repository Structure](#)
14. [2.6 Data Model](#)
15. [2.7 ADR-01 Audience Visibility and Access Control \(Salama\)](#)
16. [2.7 ADR-02 AI Feature \(Aro\)](#)
17. [2.7 ADR-03 Data Storage \(Makar\)](#)
18. [2.7 ADR-04 Collaboration and Concurrent Editing \(Omar\)](#)
19. [3 Project Management and Team Collaboration](#)
20. [4 Proof of Concept](#)
21. [4 Proof-of-Concept Sequence](#)
22. [4 Demo Script](#)

Product Scope

Source: docs/requirements/scope.md

Lead: Salama. Agreed by the whole team on 2026-09-29 (Step 1 of the brief), revised 2026-10-05 after the requirements review (see `requirements-review.md`, R-2).

Product name

CampusConnect — a social network for the MBZUAI community.

Problem

Students at MBZUAI have no single place to share campus-specific updates (events, club posts, study groups, lost & found). General social media is too broad and noisy; official channels (email, LMS announcements) are one-way and slow. Student groups prepare announcements in chat apps and copy them by hand, so drafts get lost or overwritten, and there is no shared, accountable way to deal with harmful posts.

First release (built in Assignment 2)

- **Identity:** sign-in restricted to university email accounts (mocked SSO in development).
- **Communities:** students join groups; group officers manage their group; Student Affairs marks official groups as verified.
- **Publishing:** text posts with rich-text formatting (React editor) to the campus feed or a group.
- **Collaboration:** group officers co-edit and hand off draft announcements before publishing, with conflict detection (ADR-04).
- **Audience control:** each post is campus-wide or group-only (ADR-01).
- **Interaction:** comments on posts; feed and comments update live over WebSockets.
- **Moderation:** report → hide → recorded decision → one appeal to a second moderator (ADR-01).
- **AI feature:** on-request summary of long comment threads, with every summary point linked back to the comments it came from (ADR-02).

Later releases (out of the first release, may come later)

- Events/calendar with RSVPs
- Image and file attachments
- Notifications by email or push
- Search across posts and groups
- Mobile apps

Outside the product's scope

- Direct/private messaging
- Accounts for people outside the university
- Payments or marketplace features
- Video hosting

Excluded from the Assignment 1 proof of concept

- No real authentication (author is a free-text name)
- No database (in-memory list, cleared on restart)
- No groups, comments, moderation, drafts, live updates or AI
- Plain HTML/JS instead of React; no production styling

1.1 Stakeholder Analysis

Source: docs/requirements/stakeholders.md

Lead: Salama. Drafted by the team 2026-09-29, influence added 2026-10-05.

Five stakeholder groups: four that the product serves or answers to, and one negative stakeholder the design must defend against. "Influence" describes how strongly each group shapes requirements and how that shows up in the system.

S1 — Student

- **Type:** End user (largest group).
- **Goals:** Discover and share campus updates in one place; comment and engage freely; catch up on long discussions quickly.
- **Concerns:** Over-moderation of legitimate posts; group-only posts leaking to outsiders; personal data being sent to an AI service.
- **Influence:** High on usability and performance — adoption depends on them (NFR-1, NFR-11). Low on policy: they cannot change moderation rules, but can appeal decisions (FR-14).

S2 — Group officer

- **Type:** End user with elevated rights within one group.
- **Goals:** Prepare announcements together with other officers, hand drafts off, publish quickly to the right audience (FR-7, ADR-04).
- **Concerns:** Losing a co-officer's edits; moderation delay making the group look unresponsive.
- **Influence:** Medium — drives the collaboration and audience-control requirements; can remove posts only in their own group (FR-6).

S3 — Campus moderator

- **Type:** Operations (staff or trained student volunteers).
- **Goals:** Act quickly on reports; keep evidence and a decision record for appeals (ADR-01).
- **Concerns:** Losing evidence; untraceable decisions; abuse of officer powers.
- **Influence:** High on moderation, audit and data-integrity requirements (NFR-4, NFR-5, NFR-12), which in turn drove the PostgreSQL decision (ADR-03).

S4 — University administration / Student Affairs

- **Type:** Business owner and internal regulator.
- **Goals:** Platform stays compliant with university policy; official groups are verified; no incident under the university's name; student data handled lawfully.
- **Concerns:** Impersonated groups; harmful content; privacy breaches; AI misuse of student data.
- **Influence:** Very high — can shut the platform down. Sets identity (FR-11), verification (FR-9), privacy (NFR-10) and AI data rules (ADR-02).

S5 — Bad actor / spammer (negative stakeholder)

- **Type:** Negative stakeholder.
- **Goal (adversarial):** Post spam, impersonate a group, republish removed content, or extract group-only content.
- **Influence:** Indirect but real — the reason for verified sign-in, rate limits, server-side authorization checks and the audit log (FR-11, NFR-7, NFR-12).

Where interests conflict

Tension	Sides	How the requirements/architecture resolve it
Speed vs. control	S1/S2 vs. S3/S4	Publish-then-moderate, not pre-approval: posts go live immediately and moderators act on reports within the NFR-4 target (ADR-01)
Openness vs. compliance	S1 vs. S4	Verification badge for groups (FR-9) instead of gating individual student posts
Officer autonomy vs. auditability	S2 vs. S3	Officers may remove posts in their own group only; every removal produces a moderation record (FR-6, NFR-5)
Moderator authority vs. fairness to authors	S3 vs. S1	One appeal, decided by a different moderator (FR-14, ADR-01)
AI convenience vs. privacy	S1 vs. S1/S4	Summaries only use comments the requester can already see; nothing is stored by the provider (ADR-02, NFR-10)

1.2 Functional Requirements

Source: docs/requirements/functional-requirements.md

Lead: Salama. MoSCoW priority for the Assignment 2 release. Every requirement states what the user does and how the system observably responds, so it can be tested. Stakeholder IDs (S1–S5) are from `stakeholders.md`; story IDs (US-x) from `user-stories.md`.

Requirements by area

ID	Area	The user...	The system responds by...	Priority	Source	Story
FR-11	Identity	signs in with a university email account	creating a session only for addresses on the university domain; any other address gets an error and no session	Must	S4, S5	US-1
FR-12	Communities	joins or leaves a group; a group's creator appoints officers	updating membership immediately; only members see the group's group-only posts	Must	S1, S2	US-6
FR-9	Communities	(Student Affairs) marks a group as verified	showing a "verified" badge on all of that group's posts	Should	S4	US-7
FR-1	Publishing	creates a post (1–1000 characters, rich text)	storing it and returning it with an id and timestamp; empty or too-long content is rejected with a field-level error and nothing is stored	Must	S1, S2	US-1
FR-2	Publishing	opens the campus feed	listing visible posts newest first; an empty feed shows no error	Must	S1	US-2
FR-7	Real-time collaboration	(officer) edits a shared group draft and hands it to another officer	saving each edit with a version number; a save based on an outdated version is rejected and the officer is shown the latest text to merge; other officers viewing the draft see who is editing	Should	S2	US-8
FR-13	Real-time collaboration	has the feed or a post's comments open	delivering new posts/comments they are allowed to see without a page reload	Must	S1, S2	US-5
FR-4	Audience control	chooses campus-wide or group-only when publishing	returning a group-only post only to members of that group, in every endpoint and live event	Must	S1, S2	US-6
FR-3	Interaction	comments on a post they can see (1–1000 characters)	adding the comment under the post, oldest first; empty comments are rejected	Must	S1	US-5
FR-10	Moderation	reports a post with a reason	queuing the report for moderators; the same user cannot report the same post twice	Must	S1, S3	US-3
FR-5	Moderation	(moderator) hides a reported post with a reason	removing it from all feeds while keeping the content, report and decision in a moderation record	Must	S3	US-3
FR-6	Moderation	(officer) removes a post in their own group	removing it and recording the decision; the same action on another group's post is denied	Must	S2, S3	US-4
FR-14	Moderation	(author of a hidden post) appeals once	assigning the appeal to a moderator other than the one who hid it; the outcome (restore/uphold) is recorded and shown to the author	Should	S1, S3	US-10
FR-8	AI feature	requests a summary of a thread with at least 20 comments	returning a short summary built only from comments the requester can see, labelled as AI-generated	Should	S1	US-9
FR-15	AI feature	clicks a summary point, or flags a summary as wrong	showing the source comments behind that point; a flagged summary is logged for review and can be regenerated	Should	S1, S4	US-9

The AI feature in brief (details in ADR-02)

- **Whose need:** students (S1) who open a long discussion and cannot read 50+ comments.
- **Data used:** the text of the visible comments in that thread and the authors' display names, nothing else — no email addresses, no other threads, no group-only content the requester cannot already see.
- **How the output is checked:** every summary point cites the comment ids it came from (FR-15); users can open the cited comments and flag a wrong summary; flagged summaries are reviewed by the team during Assignment 2 to measure accuracy.

Required behaviour vs. design choices

The table above is **required behaviour**. These are **design choices** recorded elsewhere, and could change without changing the requirements: - Optimistic concurrency with version numbers for drafts (ADR-04). - PostgreSQL as the data store (ADR-03). - Second-moderator appeal review (ADR-01). - Summaries generated on request rather than automatically (ADR-02).

Technology constraints (given by the course, not chosen by us)

- Backend: Python with FastAPI; live updates over WebSockets.
- Frontend: React with a rich-text editor.
- An LLM integration; paid API calls are not required, so a mock provider must be possible.

Won't (outside the product)

ID	Excluded	Reason
EX-1	Native mobile apps	Browser-only release
EX-2	Direct messages	Not core to a campus feed; raises privacy and moderation load
EX-3	Non-university users	University-only by design (FR-11)
EX-4	Payments	Not a social-feed need
EX-5	Video hosting	Storage and moderation cost out of proportion to a pilot
EX-6	Real authentication in the Assignment 1 POC	POC demonstrates only the create-post flow; see README

1.3 Non-Functional Requirements

Source: docs/requirements/non-functional-requirements.md

Lead: Salama. Every target is measurable and says how it will be checked in Assignment 2. Targets are sized for a pilot at one campus (NFR-2), not for a national-scale service.

ID	Quality	Measurable target	How it is checked	Why this target	Source
NFR-1	Performance	95% of feed requests (GET /posts , 20 posts) answer in < 500 ms with 200 concurrent users	Load test (Locust) before the A2 demo	Feeds slower than ~0.5 s feel broken; 200 concurrent ≈ busiest hour of the pilot	S1
NFR-2	Scalability	5,000 accounts, 200 groups, 50,000 posts without schema or architecture change	Seed script + NFR-1 load test on the seeded DB	Roughly the size of the MBZUAI community plus growth; assumption A-2	S4
NFR-3	Availability	99% monthly uptime between 08:00 and 22:00 Gulf time	Uptime check on /health every minute	Students use it around classes; nights can host maintenance	S1, S4
NFR-4	Live updates / moderation speed	A new post or report reaches connected clients within 2 s (p95)	Integration test with two WebSocket clients	Moderators must see reports quickly to limit harm; collaborators must see edits	S2, S3
NFR-5	Data integrity	100% of hide/remove/appeal actions produce a durable moderation record in the same transaction	Automated test that kills the request mid-action and checks no half-written state	Decisions without records cannot be appealed or audited	S3
NFR-6	Accessibility	WCAG 2.1 AA on sign-in, feed, post, comment and draft screens	axe-core in CI plus one manual keyboard-only pass	University digital services are expected to be accessible	S1, S4
NFR-7	Security	100% of write endpoints reject unauthenticated or unauthorised requests (401/403); no secrets in the repo or browser bundle	Authorization test matrix (role × endpoint); secret scan in CI	Group-only content and moderator powers are the main attack targets (S5)	S4, S5
NFR-8	Concurrency correctness	0 silent overwrites: every stale draft save returns 409	Test with two clients saving the same draft version	Officers must not lose each other's work (FR-7)	S2
NFR-9	Recovery	Data loss ≤ 15 minutes (RPO) and service restored within 4 hours (RTO)	Restore drill from backup once before the A2 demo	A lost moderation record or announcement is worse than short downtime	S3, S4
NFR-10	Privacy	0 group-only posts returned to non-members; LLM requests contain only comment text and display names; a user's data is deleted within 30 days of a deletion request	Authorization tests; logged LLM payloads reviewed in tests	Students' trust and university data rules	S1, S4
NFR-11	Usability	4 of 5 first-time test users create a post and a comment in under 2 minutes without help	Short hallway test with 5 students in A2	If posting is harder than a chat app, students stay in chat apps	S1
NFR-12	Auditability	100% of moderation actions, appeals, role changes and verifications are logged with actor, time and reason, kept 1 year, not editable through the API	Test that each action writes an audit row; no update/delete endpoint exists for it	Appeals and university complaints need a trustworthy history	S3, S4

2.1 Architectural drivers (ranked)

Lead: Salama, with technical input from Makar (data, backend) and Omar (frontend, real-time).

Rank	Driver	How it shaped the design
1	Audience control & privacy (FR-4, NFR-7, NFR-10)	All visibility checks live in one backend component (Access Policy, see component diagram) used by REST <i>and</i> WebSocket paths; the browser is never trusted to filter.
2	Data integrity & auditability (NFR-5, NFR-12)	Relational database with foreign keys and transactions (ADR-03); moderation records and audit events are append-only rows.
3	Concurrent collaboration (FR-7, NFR-8, NFR-4)	Version-checked draft saves plus a WebSocket channel for presence and change notices (ADR-04), instead of a CRDT engine.
4	Trustworthy AI output (FR-8, FR-15)	Summaries cite comment ids; LLM behind an adapter so a mock can be used and payloads can be tested (ADR-02).
5	Performance & scalability (NFR-1, NFR-2)	Paginated feed, indexed <code>created_at / group_id</code> ; one FastAPI service is enough for the pilot, so no microservices.
6	Availability & recovery (NFR-3, NFR-9)	Managed Postgres backups; stateless API so it can be restarted or duplicated.
7	Accessibility & usability (NFR-6, NFR-11)	Affects frontend component choice (accessible rich-text editor), not the overall structure.

1.4 User Stories and Scenarios

Source: docs/requirements/user-stories.md

Lead: Aro. Ten stories from four stakeholder groups, covering normal use, failures, exceptions and policy decisions. Each acceptance criterion is a Given/When/Then check.

Story	Stakeholder	Part of the system	Covers
US-1	Student (S1)	Identity, publishing	normal use, validation failure
US-2	Student (S1)	Feed	normal use, empty case
US-3	Moderator (S3)	Moderation	policy, duplicate report exception
US-4	Group officer (S2)	Moderation	policy, permission failure
US-5	Student (S1)	Interaction, live updates	normal use, failure, reconnect
US-6	Student / officer (S1, S2)	Communities, audience control	policy, privacy
US-7	Student Affairs (S4)	Communities	policy, permission failure
US-8	Group officer (S2)	Real-time collaboration	concurrency exception, failure
US-9	Student (S1)	AI feature	normal use, AI failure, output check
US-10	Student (S1) + moderator (S3)	Moderation appeals	policy, exception

US-1: Sign in and create a post

As a **student**, I want to sign in with my university account and post an update, so that the campus sees it and knows it comes from a real student. - **Given** I sign in with an @mbzuai.ac.ae account, **when** sign-in completes, **then** I see the feed. - **Given** I sign in with a non-university email, **when** sign-in completes, **then** I see "university accounts only" and no session is created. - **Given** I am signed in, **when** I submit a post of 1–1000 characters, **then** it appears at the top of the feed without a page reload. - **Given** I submit empty, whitespace-only or 1001+ character content, **when** I press Post, **then** I see a field error and no post is stored (contract: 422).

US-2: View the campus feed

As a **student**, I want to see recent posts newest first, so that I stay informed. - **Given** posts exist, **when** I open the feed, **then** they are ordered newest first, 20 per page. - **Given** no posts exist, **when** I open the feed, **then** I see an empty feed and no error.

US-3: Report and hide a harmful post

As a **campus moderator**, I want reported posts to reach me quickly and to hide them with a reason, so that harmful content stops spreading but can still be reviewed. - **Given** a student reports a post, **when** I have the moderation queue open, **then** the report appears within 2 seconds. - **Given** the same student reports the same post again, **when** they submit, **then** they see "already reported" and no second report is stored. - **Given** I hide a post with a reason, **when** anyone else loads the feed, **then** the post is gone, but its content, the reports and my decision are kept in a moderation record.

US-4: Group officer removes a post in their group

As a **group officer**, I want to remove posts in my own group, so that I can keep it on topic. - **Given** a post belongs to my group, **when** I remove it, **then** it disappears from the group feed and a moderation record names me as the decider. - **Given** a post belongs to a group where I am not an officer, **when** I try to remove it, **then** I get "not allowed" (403) and the post is unchanged.

US-5: Comment and see replies live

As a **student**, I want to comment on a post and see others' comments appear, so that I can take part in the discussion. - **Given** I can see a post, **when** I submit a 1–1000 character comment, **then** it appears under the post (oldest first) for me and, within 2 seconds, for others viewing the post. - **Given** I submit an empty comment, **when** I press Send, **then** I see an error and nothing is stored. - **Given** my connection drops and returns, **when** the page reconnects, **then** comments posted meanwhile are shown (refetched), with none missing or duplicated.

US-6: Choose who sees my post

As a **student or group officer**, I want to post either campus-wide or to my group only, so that internal group announcements stay inside the group. - **Given** I am a member of group G, **when** I publish a group-only post to G, **then** members of G see it and non-members do not — in the feed, by direct link (404) and in live events. - **Given** I am not a member of group G, **when** I try to post to G, **then** I get 403. - **Given** I leave group G, **when** I reload the feed, **then** G's group-only posts are no longer shown.

US-7: Student Affairs verifies a group

As a **Student Affairs staff member**, I want to mark official groups as verified, so that students can trust who is posting. - **Given** an unverified group, **when** I verify it, **then** its posts show a "verified" badge and an audit event records who verified it. - **Given** a user without the Student Affairs role, **when** they call the verify action, **then** they get 403.

US-8: Two officers prepare an announcement together

As a **group officer**, I want to edit a shared draft with my co-officers and hand it over, so that we publish one agreed announcement without losing anyone's changes. - **Given** two officers have draft version 7 open, **when** both save, **then** the first save creates version 8 and the second gets a conflict with the latest text to merge — no edit is lost. - **Given** another officer opens the draft, **when** I am viewing it, **then** I see their name in the presence list within 2 seconds. - **Given** the database fails while publishing, **when** I press Publish, **then** I see an error, no post is created, and the draft is unchanged so I can retry.

US-9: Summarise a long thread and check it

As a **student**, I want a short summary of a long discussion that I can check against the real comments, so that I can catch up without being misled. - **Given** a thread with ≥ 20 comments I can see, **when** I click "Summarise", **then** I get 3–6 points labelled "AI-generated", each linking to the comments it is based on. - **Given** a thread with fewer than 20 comments, **when** I look at it, **then** the Summarise button is not offered. - **Given** the AI service is unavailable, **when** I click "Summarise", **then** I see "summary unavailable, try later" within 10 seconds and the thread still works. - **Given** I think a point is wrong, **when** I click "Flag", **then** the flag is stored for review.

US-10: Appeal a moderation decision

As a **student whose post was hidden**, I want to appeal once, so that a mistake can be corrected. - **Given** my post was hidden, **when** I submit an appeal, **then** it is assigned to a moderator other than the one who hid it. - **Given** I already appealed this decision, **when** I try again, **then** I am told only one appeal is allowed. - **Given** the moderator who hid my post tries to decide the appeal, **when** they submit, **then** the system refuses (403). - **Given** the appeal is decided "restored", **when** students load the feed, **then** the post is visible again.

1.5 Validation and Traceability

Source: docs/requirements/requirements-review.md

Lead: Aro. Feasibility checked by Makar and Omar; requirement wording owned by Salama.

How we reviewed the requirements

1. **Scope walkthrough (2026-09-29, all four):** agreed users, features, visibility rules, moderation policy and AI feature before writing anything (Step 1 of the brief).
2. **Checklist per requirement:** for every FR/NFR we asked: Is it observable behaviour? Can it be tested, and how? Which stakeholder needs it? Is it a requirement or a design choice? Does it conflict with another requirement? Vague words ("fast", "secure", "quickly") were replaced by numbers.
3. **Feasibility check (Makar, Omar):** each requirement mapped to a component in `diagrams/c4-component.md` and to an endpoint or event in `api-contract.md`.
4. **Consistency review (2026-10-05):** compared requirements, diagrams, contract, ADRs and the running POC side by side (Step 5 of the brief). Findings below.

Contradictions and unclear wording we resolved

#	Problem found	Resolution
R-1	ER diagram used visibility <code>public</code> , ADR-01 said "campus-wide"	One vocabulary everywhere: <code>campus</code> / <code>group</code>
R-2	Scope listed live updates and rich text as <i>future</i> , but Assignment 2 requires them and NFR-4 depends on WebSockets	Moved into the first release (<code>scope.md</code>)
R-3	ADR-01 requires appeals, but the ER model had no field for them	<code>appeal_*</code> fields and a CHECK (second moderator $\neq$ first) added to <code>MODERATION_RECORD</code>
R-4	NFR-4 said reports reach moderators "within 1 minute" while the design pushes them live	Single target: live events within 2 s (p95)
R-5	FR-9 (group verification) had no story and no owner	US-7 added; policy owned by ADR-01
R-6	"Group" and "officer" were undefined	Defined through <code>GROUP_MEMBERSHIP</code> with role <code>member</code> / <code>officer</code>
R-7	US-1 says empty posts are rejected, but the POC accepted content of only spaces	Backend trims before validating; test added
R-8	No requirement covered identity, joining groups, live updates or appeals, though the brief lists them	FR-11 ... FR-15 and US-8 ... US-10 added

Assumptions

ID	Assumption	If wrong
A-1	University context is MBZUAI; a valid account is an <code>@mbzuai.ac.ae</code> address from the university identity provider	Change the domain list in Auth config
A-2	Pilot size $\approx$ 5,000 accounts and 200 groups (NFR-2); not confirmed by Student Affairs	Re-run load test with the real figure
A-3	At least two active moderators at any time (appeals, ADR-01)	Appeals escalate to Student Affairs (R6)
A-4	Group verification is a manual decision by Student Affairs	—
A-5	A free or mock LLM is acceptable for Assignment 2 (brief: paid calls not required)	Adapter allows another provider

ID	Assumption	If wrong
A-6	Sending comment text and display names to the LLM provider is acceptable to the university if nothing is retained by the provider	Switch to a self-hosted model behind the same adapter

Open questions

ID	Question	Who decides
Q-1	Must the 99% availability target (NFR-3) hold during grading, or only during the demo? Assumed: during the Assignment 2 demo period	Teaching team
Q-2	How is the first Student Affairs account created? Planned: a seed script run by an admin, not an API endpoint	Team (Makar) in A2
Q-3	Can students report anonymously to the author? Planned: reporter is hidden from the author, visible to moderators	Student Affairs
Q-4	How long are hidden posts kept after an appeal is decided? Assumed: 1 year with the audit log (NFR-12)	Student Affairs

Traceability matrix

Stakeholders: `stakeholders.md` · Requirements: `functional-requirements.md`, `non-functional-requirements.md`
 · Stories: `user-stories.md` · Components: `diagrams/c4-component.md` · ADRs: `adr/`.

Stakeholder need	FR	NFR	Story	Architecture components	ADR	Verified by
S4, S5: only real university members	FR-11	NFR-7	US-1	Auth, University sign-in	ADR-01	API auth tests (A2)
S1: share an update	FR-1	NFR-1, NFR-11	US-1	Web App, Posts & Comments router, Repositories	—	POC: <code>test_create_post</code> , <code>test_whitespace_only_content_returns_422</code> , <code>test_content_over_1000_chars_returns_422</code> , browser demo
S1: see what is happening	FR-2	NFR-1, NFR-2	US-2	Posts & Comments router, Repositories	ADR-03	POC: <code>test_list_posts_newest_first</code> , <code>test_list_posts_empty</code> ; load test (A2)
S1: discuss posts	FR-3	NFR-1	US-5	Posts & Comments router	—	API tests (A2)
S1, S2: control who sees content	FR-4	NFR-10	US-6	Access Policy, Realtime hub, Repositories	ADR-01	Role × endpoint matrix (A2)
S1, S2: belong to communities	FR-12	NFR-10	US-6	Groups router, Access Policy	ADR-01	API tests (A2)
S4: trusted official groups	FR-9	NFR-12	US-7	Groups router, Access Policy	ADR-01	API tests (A2)
S2: prepare announcements together	FR-7	NFR-8, NFR-4	US-8	Drafts service, Realtime hub, Database	ADR-04, ADR-03	Two-client conflict test (A2)
S1, S2: see changes without reloading	FR-13	NFR-4	US-5	Realtime hub, Access Policy	ADR-04	Two WebSocket client test (A2)
S3: act on harmful posts	FR-10,	NFR-4, NFR-	US-3	Moderation service,	ADR-01,	Transaction test (A2)

Stakeholder need	FR	NFR	Story	Architecture components	ADR	Verified by
	FR-5	5, NFR-12		Realtime hub, Database	ADR-03	
S2, S3: officers manage own group	FR-6	NFR-5, NFR-7	US-4	Moderation service, Access Policy	ADR-01	Role × endpoint matrix (A2)
S1, S3: fair decisions	FR-14	NFR-12	US-10	Moderation service, Database CHECK	ADR-01	API tests (A2)
S1: catch up on long threads	FR-8	NFR-10	US-9	Summary service, LLM Provider, Access Policy	ADR-02	MockSummarizer tests (A2)
S1, S4: AI output can be checked	FR-15	NFR-12	US-9	Summary service, Web App summary panel	ADR-02	Citation-check tests (A2)
S3, S4: recover from failures	—	NFR-3, NFR-9	—	Database backups, stateless API	ADR-03	Restore drill (A2)
S1, S4: accessible to all students	—	NFR-6	—	Web App	—	axe-core in CI (A2)

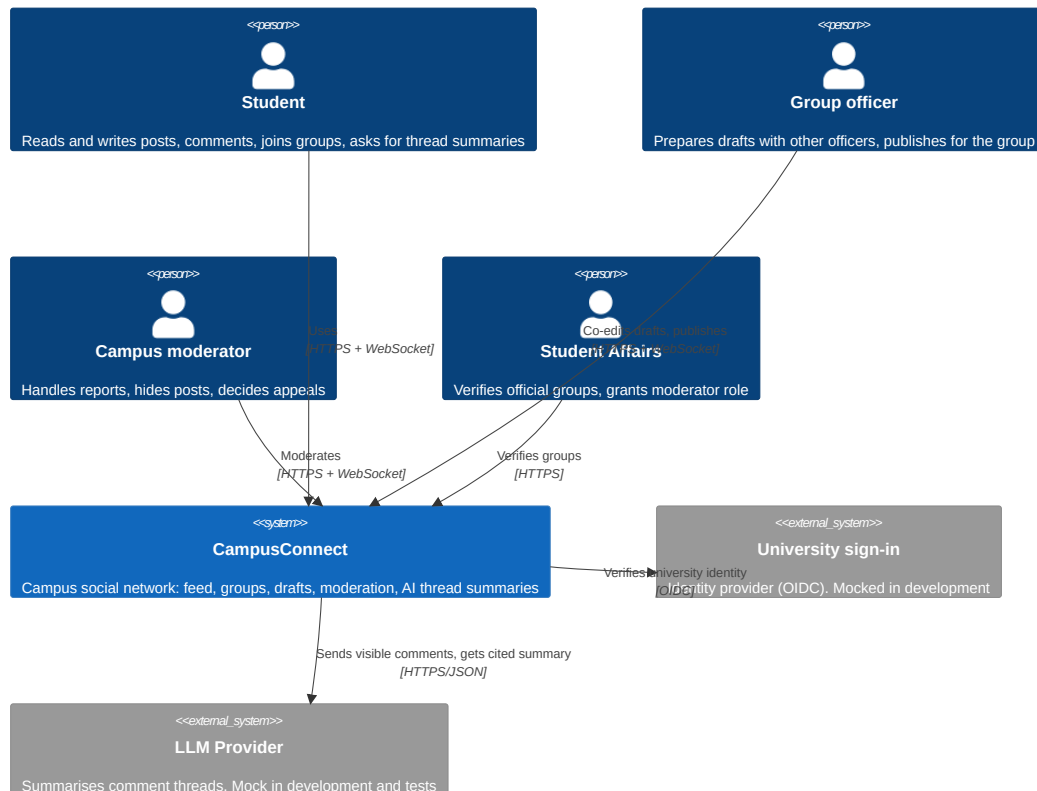
Every FR has at least one story; every story traces to at least one FR, component and verification method.

2.2 C4 — Level 1: System Context

Source: docs/architecture/diagrams/c4-context.md

Lead: Omar. Editable Mermaid source; renders on GitHub and in Report.pdf.

CampusConnect — System Context



Explanation

- **System boundary:** CampusConnect is the only system we build. Everything inside it is shown in the container diagram.
- **Users:** the four human stakeholder groups from stakeholders.md (S1–S4). The bad actor (S5) is not drawn as a user; the design defends against them through identity (FR-11) and authorization (ADR-01).
- **External systems:** the university identity provider, so only university accounts can sign in (FR-11), and an LLM provider for thread summaries (FR-8). Both sit behind backend adapters, so mocks replace them in development and in the POC — no paid calls needed.
- **Communication:** browsers use HTTPS for requests and a WebSocket for live updates; only the backend talks to the external systems, so no secret reaches the browser.

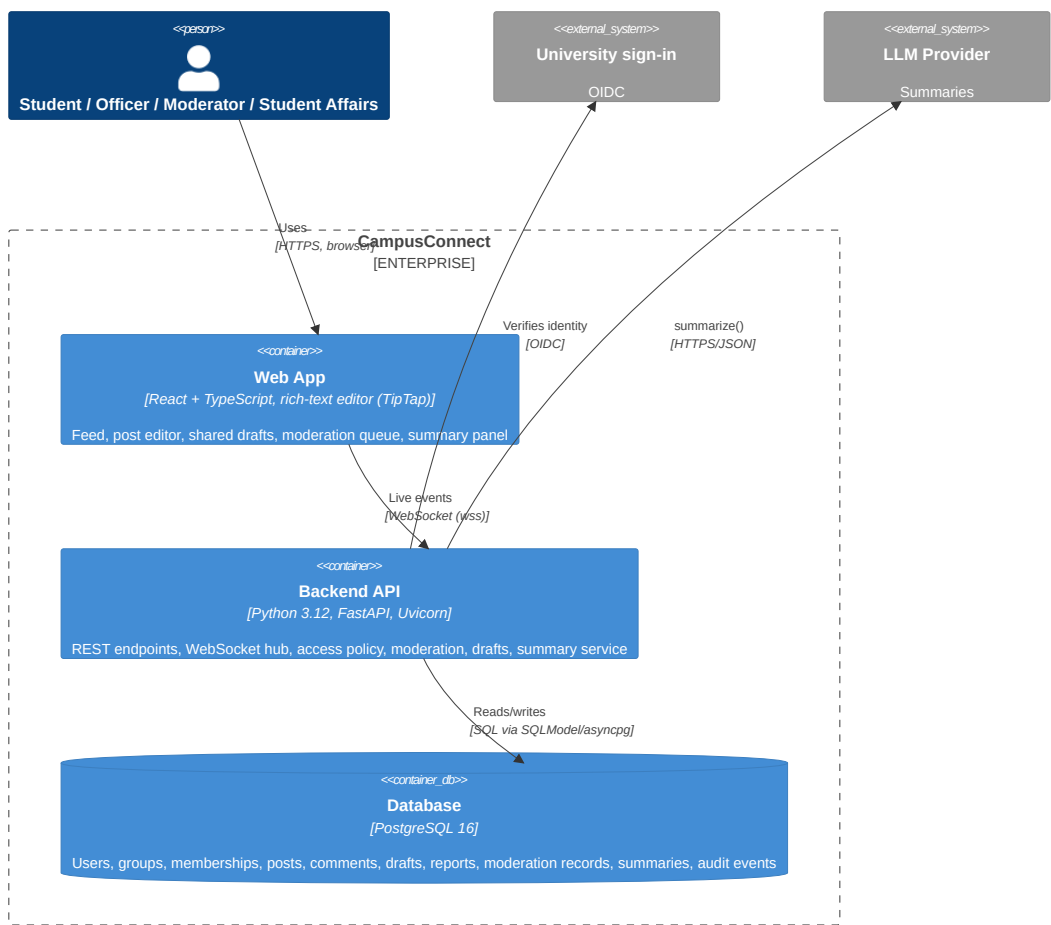
2.2 C4 — Level 2: Containers

Source: docs/architecture/diagrams/c4-container.md

Lead: Omar, backend details from Makar.

Assignment 2 target

CampusConnect — Containers (Assignment 2 target)



Explanation

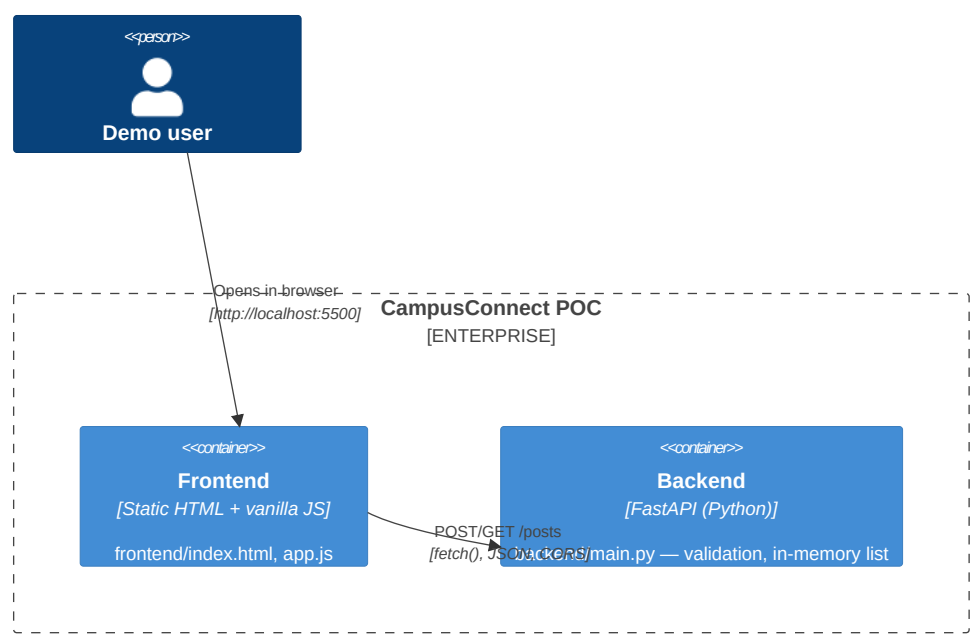
- Web App** — single-page React app served as static files. Only renders and sends requests; it holds no secrets and makes no access decisions.
- Backend API** — one FastAPI application serving REST and WebSocket endpoints. Its internal structure is in `c4-component.md`.
- Database** — PostgreSQL, the only place state is kept (ADR-03). The API process is stateless apart from open WebSocket connections, so it can be restarted (NFR-3, NFR-9).
- Communication:** HTTPS/JSON for all writes and reads, WebSocket only for push notifications, SQL between API and database. External calls go out only from the API.

Technology choices and Assignment 2 constraints

Choice	Why	Assignment 2 constraint it meets
FastAPI	Async, Pydantic validation, built-in WebSockets and OpenAPI export; same framework as the POC	Python/FastAPI backend with WebSockets
WebSocket hub inside the API (not a separate service)	One deployable unit at pilot scale; shares the Access Policy code directly. If it must scale out, add Redis pub/sub between API instances	WebSockets
React + TipTap	Accessible, extensible rich-text editor that outputs structured JSON we can sanitise server-side	React frontend with rich text
PostgreSQL	Relational data with constraints and transactions (ADR-03)	—
LLM behind an adapter with a mock	Swap providers, test without paid calls, check citations (ADR-02)	LLM integration

Assignment 1 proof of concept (what runs today)

CampusConnect POC — Containers (Assignment 1, as implemented)

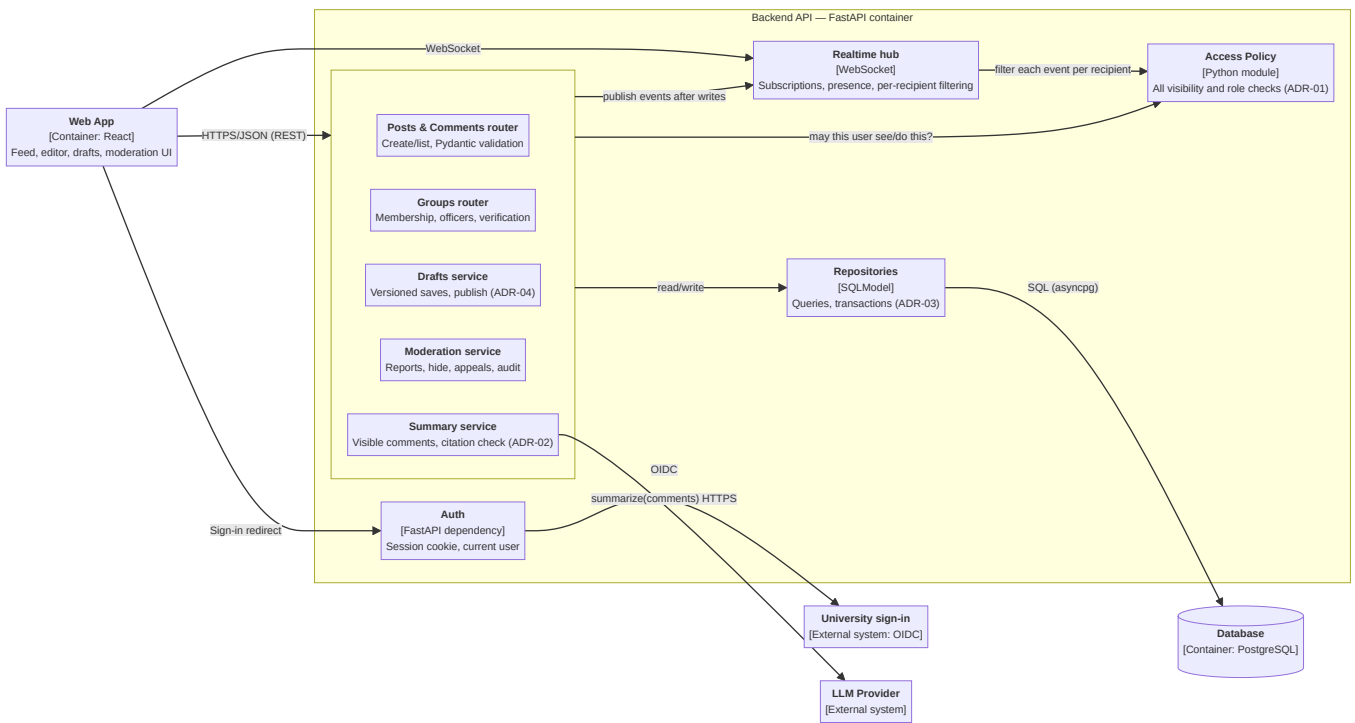


The POC keeps the same frontend/backend split and the same contract style as the target, but replaces React with plain JavaScript and PostgreSQL with an in-memory list (see `docs/poc.md`).

2.2 C4 — Level 3: Components of the Backend API

Source: docs/architecture/diagrams/c4-component.md

Lead: Makar (backend component diagram, as assigned in the brief). Selected container: **Backend API** — it holds every rule the requirements care about (authorization, validation, moderation, concurrency, AI checks), so it is the container most worth decomposing.



Responsibilities

Component	Responsible for	Not responsible for
Auth	Turning the identity provider's answer into a session; rejecting non-university emails (FR-11)	Deciding what a user may see
Access Policy	Every "may this user see/do this?" decision (FR-4, FR-6, FR-14, NFR-10)	Storing data
Posts & Comments router	Request validation (Pydantic), creating/listing posts and comments	Visibility rules (asks Access Policy)
Groups router	Membership, officer appointment, verification (FR-9, FR-12)	—
Drafts service	Version check on save, conflict response with latest draft, publish (FR-7, NFR-8)	Live presence (Realtime hub)
Moderation service	Reports, hide/remove, appeals, writing moderation and audit rows in one transaction (NFR-5, NFR-12)	—
Summary service	Selecting visible comments, calling the LLM adapter, dropping uncited points (FR-8, FR-15)	Choosing the model vendor (adapter config)
Realtime hub	WebSocket connections, subscriptions, presence, delivering events (FR-13, NFR-4)	Accepting writes — all writes go through REST
Repositories	SQL queries, transactions, optimistic-lock UPDATE	Business rules

The proof of concept (backend/main.py) contains a minimal version of the Posts router only: Pydantic validation and an in-memory list in place of Repositories.

The five boxes in the middle row are drawn together because they share the same three dependencies (Access Policy, Realtime hub, Repositories); the table above lists each one's responsibility separately.

2.3 Behaviour and Design Decisions

Source: docs/architecture/behaviour-and-design.md

Lead: Omar (coordination, frontend), backend details by Makar. Component names refer to `diagrams/c4-component.md`; the collaborative sequence diagram is `diagrams/sequence-draft-handoff.md`.

Audience and authorization rules

- Each post is `campus` or `group` (ADR-01). The **Access Policy** component decides every read and write; routers, the Realtime hub and the Summary service all call it.
- Feed queries filter in SQL by the caller's memberships, so hidden or group-only rows never leave the database for the wrong user. A post the caller may not see returns **404**, not 403, so its existence is not revealed.
- The frontend hides buttons a user cannot use, but this is convenience only; the backend rechecks everything (NFR-7).

Publishing and collaboration

- Students publish directly: `POST /posts` validates (Pydantic), stores, then notifies the hub.
- Officers prepare group announcements as shared **drafts**. Saves carry a version number; stale saves get **409** with the latest text to merge (ADR-04, sequence diagram). Publishing is a single transaction that creates the post and closes the draft.

Live updates

- One WebSocket per browser tab. Clients subscribe to `feed`, `post:<id>` or `draft:<id>`.
- After any successful write, the service publishes an event to the hub; the hub asks the Access Policy for each subscriber before sending, so live events obey the same rules as REST.
- Events are notifications, not the source of truth: on reconnect the client refetches over REST. This keeps writes, validation and authorization in one place.

Moderation and appeals

- Any user can report a post once (FR-10). Reports are pushed live to moderators (NFR-4).
- A moderator (any post) or officer (own group) hides/removes a post. The status change, the `MODERATION_RECORD` and an `AUDIT_EVENT` are written in one transaction (NFR-5, NFR-12). Content is kept, not deleted.
- The author can appeal once; a *different* moderator decides (database CHECK enforces it). Restored posts reappear in feeds through a normal `post.created`-style event.

AI feature

- On request, for threads with ≥ 20 comments, the **Summary service** collects only the comments the requester can see, sends id + display name + text to the LLM adapter, and asks for points that each cite comment ids (ADR-02).
- Points without valid citations are dropped. The UI labels the result as AI-generated, links each point to its comments, and offers "flag as wrong" (FR-15).
- Summaries are stored with the comment count; when the thread grows by 10+ comments the UI offers to regenerate.

Responsibility for each behaviour

Behaviour	Responsible part
Input validation (lengths, required fields)	Backend routers (Pydantic); browser checks are only for convenience
Sign-in, session	Auth component + university identity provider
Who can see / do what	Access Policy
Draft conflict detection	Drafts service + database conditional UPDATE

Behaviour	Responsible part
Pushing live changes	Realtime hub
Moderation records, appeals, audit	Moderation service, database transaction
Building and checking summaries	Summary service; LLM provider only generates text
Rendering rich text safely	Frontend editor + backend sanitising stored HTML/JSON

Failures we planned for

Failure	Planned response	Requirement
LLM slow, down or returns uncited text	8 s timeout, one retry, then 503 "summary unavailable"; uncited points dropped; rest of the app unaffected	FR-8, FR-15
WebSocket disconnect	Reconnect with backoff, refetch over REST	FR-13, NFR-4
Two officers save the same draft	409 with latest content, manual merge	NFR-8
Database unavailable	API returns 503; <code>/health</code> reports it; restore from backups within RTO	NFR-3, NFR-9
Crash in the middle of a moderation action	Transaction rolls back, no half-written decision	NFR-5
Spam / flooding	Rate limit 10 posts/min per user (429), report flow	S5
Script injection in posts	Frontend renders text, never raw HTML; rich text sanitised server-side	NFR-7
Only one moderator available for an appeal	Appeal escalates to Student Affairs	R6, ADR-01

Trade-offs

- **Optimistic versions instead of a CRDT** (ADR-04): far simpler and testable, but two people cannot type in the same sentence simultaneously; acceptable for announcement drafts.
- **One FastAPI service instead of microservices**: simpler to build and debug for 4 people; limit is horizontal scaling of WebSockets, solvable later with Redis pub/sub.
- **Publish-then-moderate**: fast for students (S1), but harmful content can be visible until reported; mitigated by live report delivery and the moderation target (NFR-4).
- **Summaries on request, not automatic**: lower LLM cost and less unsolicited AI text, but users must click once.

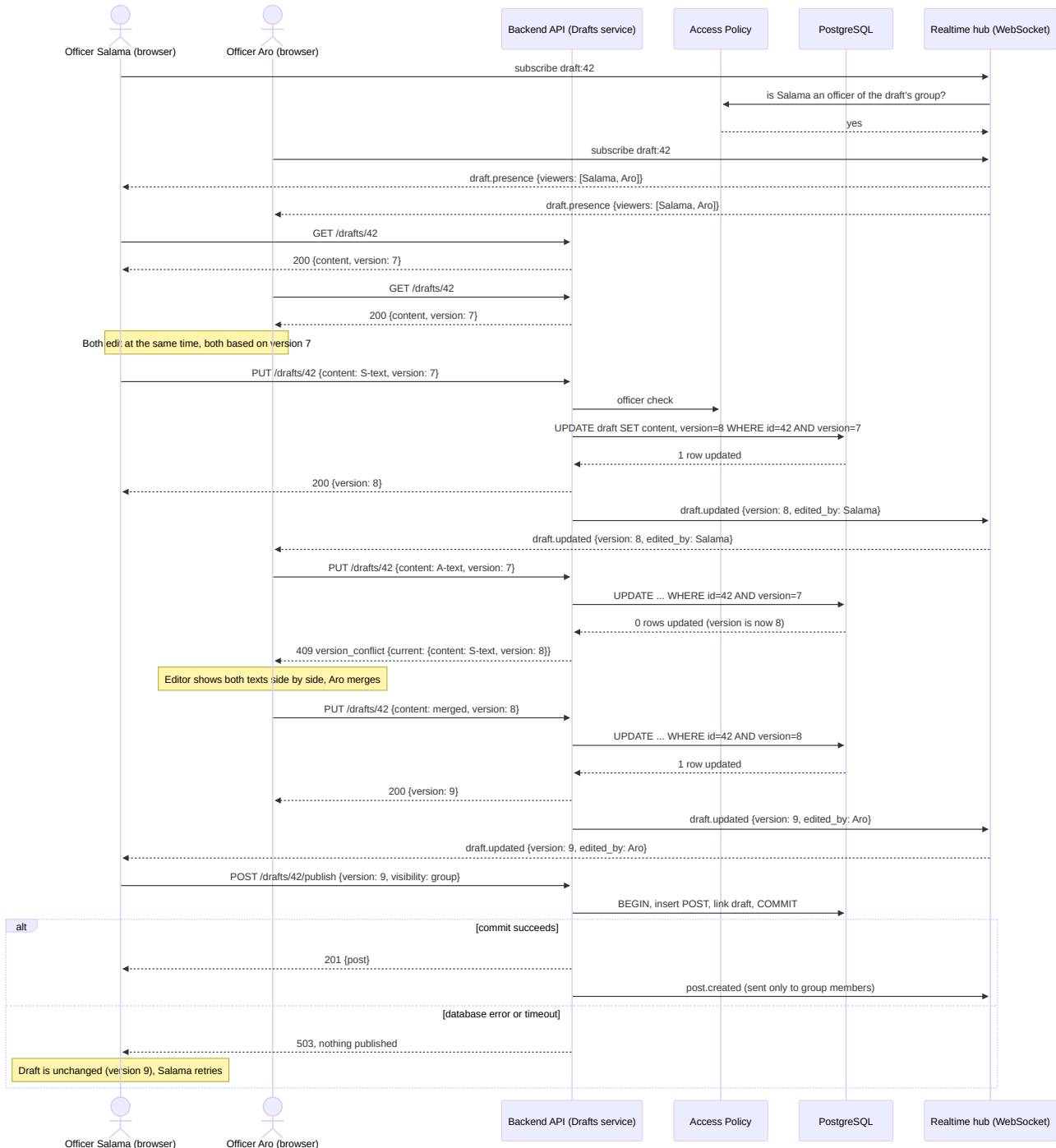
Developing and testing components separately, and together

- **Contract first**: the API contract (2.4) is agreed before coding; FastAPI's `openapi.json` is exported and the frontend client is generated from it, so contract drift fails the build.
- **Separately**: backend unit tests mock the database and the LLM adapter (`MockSummarizer`); Access Policy has its own role × resource test table; frontend components are tested against a mocked API client.
- **Together**: API tests run against a real Postgres in Docker; Playwright end-to-end tests drive the real frontend and backend, including two browser contexts for the draft conflict.
- **Already in Assignment 1**: `backend/test_main.py` (8 tests) checks the POC contract, and the create-post flow was checked end-to-end in a real browser.

2.3 UML Sequence Diagram — Two Officers Co-edit and Publish a Draft

Source: docs/architecture/diagrams/sequence-draft-handoff.md

Lead: Omar, backend steps by Makar. The collaborative interaction required by 2.3: officers Salama and Aro of the same group prepare one announcement (FR-7, US-8, ADR-04).



What happens when users act concurrently

- Both officers can type at the same time; nothing is locked. Saves are checked against the version the officer started from. The first save wins; the second gets **409** with the current text, so no edit is silently lost (NFR-8).
- Presence events show who else is in the draft, so most conflicts are avoided before they happen.
- If both press **Publish** at once, both requests carry version 9: the first creates the post and marks the draft published; the second gets **409 (already published)** — the announcement is never posted twice.

What happens when an operation fails

Failure	Behaviour
WebSocket drops	Client reconnects with backoff and refetches the draft over REST; edits are never sent over the socket, so none are lost
Save request times out	Client keeps the text locally and retries with the same version; a duplicate is harmless because the version check rejects it (409 with the same content)
Database error during publish	Transaction rolls back; no post exists, draft unchanged, client gets 503 and can retry
Officer loses officer role mid-edit	Next save returns 403; the draft stays for the remaining officers
Officer opens the editor while offline	Editor is read-only until the latest version is fetched

2.4 Interfaces

Source: docs/architecture/api-contract.md

Lead: Makar (backend), agreed with Omar (frontend) on 2026-09-29 before coding (Step 3 of the brief). Part A is implemented and tested in the proof of concept. Part B is the Assignment 2 contract the architecture is designed around.

Part A — Proof-of-concept contract (implemented)

Base URL `http://localhost:8000` . JSON over HTTP. No authentication in the POC.

POST /posts — create a post

Request

```
{ "author": "Omar", "content": "Study group for AI1220 at 6pm, library room 2" }
```

Field	Rule
author	string, required, 1–100 characters after trimming spaces
content	string, required, 1–1000 characters after trimming spaces

201 Created

```
{
  "id": "6f1c2a4e-1b7d-4c55-9a0e-2f8f3b9d7c11",
  "author": "Omar",
  "content": "Study group for AI1220 at 6pm, library room 2",
  "created_at": "2026-10-05T09:30:12.123456Z"
}
```

422 Unprocessable Entity — missing, empty, whitespace-only or too-long field. Nothing is stored. This is FastAPI/Pydantic v2's standard shape; the frontend shows `loc[-1]: msg` .

```
{
  "detail": [
    {
      "type": "missing",
      "loc": ["body", "content"],
      "msg": "Field required",
      "input": { "author": "Omar" }
    }
  ]
}
```

Other `type` values the POC can return: `string_too_short` (empty/whitespace), `string_too_long` (> limit), `string_type` (not a string).

GET /posts — list posts

200 OK — array of the post objects above, newest first. `[]` when there are no posts.

GET /health

200 OK — `{ "status": "ok" }`

Verified by `backend/test_main.py` (8 tests) and by the browser demo.

Part B — Assignment 2 interfaces (planned)

Authentication and authorization rules

- The browser signs in through the university identity provider (OIDC; a mock provider in development). The backend issues a session cookie (`HttpOnly` , `Secure` , `SameSite=Lax`). No token is stored in JavaScript.
- Every request except `/health` and the sign-in callback requires a session → otherwise **401**.
- Authorization is decided by the Access Policy component (ADR-01), never by the browser:

Role	Can
student	read campus posts and posts of groups they belong to; post, comment, report, appeal own hidden posts, request summaries
officer (per group)	everything a student can, plus edit that group's drafts, publish as the group, remove posts in that group
moderator	hide any post, decide appeals on decisions they did not make
student_affairs	verify groups, grant/revoke moderator role

Error format

- **422** keeps FastAPI's validation shape from Part A.
- All other errors: `{ "detail": "<human-readable message>", "code": "<machine code>" }`

Status	When	Example code
401	no or expired session	<code>not_authenticated</code>
403	authenticated but not allowed	<code>not_group_member</code> , <code>not_officer</code> , <code>same_moderator</code>
404	resource missing or not visible to the caller (no existence leak)	<code>not_found</code>
409	stale draft version, duplicate report, second appeal	<code>version_conflict</code> , <code>already_reported</code> , <code>appeal_exists</code>
429	more than 10 posts or 30 comments per user per minute	<code>rate_limited</code>
503	LLM provider unavailable or timed out (8 s)	<code>summary_unavailable</code>

REST endpoints

Method & path	Request body	Success	Who	Failures	FR
GET <code>/me</code>	—	200 user + roles + memberships	any	401	FR-11
GET <code>/posts?group_id=&before=&limit=20</code>	—	200 visible posts, newest first	any	401, 403 (group not joined)	FR-2, FR-4
POST <code>/posts</code>	<code>{content, visibility: "campus" "group", group_id?}</code>	201 post	any; group needs membership	401, 403, 422, 429	FR-1, FR-4
GET <code>/posts/{id}/comments</code>	—	200 comments, oldest first	can see post	401, 404	FR-3
POST <code>/posts/{id}/comments</code>	<code>{content}</code>	201 comment	can see post	401, 404, 422, 429	FR-3

Method & path	Request body	Success	Who	Failures	FR
POST /groups/{id}/members	—	201 membership	any	401, 404, 409	FR-12
DELETE /groups/{id}/members/me	—	204	member	401, 404	FR-12
POST /groups/{id}/verify	—	200 group	student_affairs	401, 403	FR-9
POST /groups/{id}/drafts	{content}	201 draft {id, version: 1}	officer	401, 403, 422	FR-7
PUT /drafts/{id}	{content, version}	200 draft with version + 1	officer of group	401, 403, 409 + latest draft	FR-7
POST /drafts/{id}/publish	{version, visibility}	201 post	officer of group	401, 403, 409	FR-7
POST /posts/{id}/reports	{reason}	201 report	can see post	401, 404, 409, 422	FR-10
POST /posts/{id}/hide	{reason}	200 moderation record	moderator, or officer of the post's group	401, 403, 404	FR-5, FR-6
POST /moderation/{id}/appeal	{statement}	201 appeal pending	author of the post	401, 403, 409	FR-14
POST /moderation/{id}/appeal/decision	{outcome: "restored" "upheld", note}	200	moderator ≠ original decider	401, 403 (same_moderator)	FR-14
POST /posts/{id}/summary	—	200 summary (below)	can see post	401, 404, 422 (< 20 comments), 503	FR-8
POST /summaries/{id}/flag	{reason}	201	can see post	401, 404	FR-15

409 on a stale draft save returns the current draft so the client can merge:

```
{ "detail": "Draft changed since you opened it", "code": "version_conflict",
  "current": { "id": "...", "version": 7, "content": "...", "last_edited_by": "Aro" } }
```

Summary response

```
{
  "id": "...", "post_id": "...", "generated_at": "2026-10-05T10:00:00Z",
  "label": "AI-generated summary – check the linked comments",
  "points": [
    { "text": "Most people prefer Thursday for the meetup.", "source_comment_ids": ["c12", "c19", "c31"] }
  ],
  "comment_count": 54
}
```

WebSocket events — wss://.../ws

The client connects with its session cookie and subscribes; the server sends only events the user may see (filtered by Access Policy per recipient).

```
// client → server
{ "type": "subscribe", "channel": "feed" }
{ "type": "subscribe", "channel": "post:<post_id>" }
{ "type": "subscribe", "channel": "draft:<draft_id>" } // officers of that group only

// server → client
{ "type": "post.created", "post": { ...post... } }
{ "type": "post.hidden", "post_id": "..." }
{ "type": "comment.created", "post_id": "...", "comment": { ...comment... } }
{ "type": "draft.updated", "draft_id": "...", "version": 8, "edited_by": "Salama" }
{ "type": "draft.presence", "draft_id": "...", "viewers": ["Salama", "Aro"] }
{ "type": "report.created", "post_id": "..." } // moderators only
{ "type": "error", "code": "forbidden_channel" }
```

Live events are notifications only. Writes always go through REST, so validation and authorization live in one place. After a reconnect the client refetches over REST, so a missed event never leaves stale data on screen.

External service — LLM provider (behind the Summary adapter)

```
summarize(comments: list[{id, author_display_name, text}]) -> list[{text, source_comment_ids}]
```

- Implementations: `MockSummarizer` (dev/tests, deterministic) and a real provider client.
- Timeout 8 s, one retry, then 503 `summary_unavailable` to the client.
- Points that cite no comment id, or an id that was not in the input, are dropped before the response is returned (ADR-02).
- The API key lives only in the backend's environment (see `repository-structure.md`).

2.5 Code and Repository Structure

Source: docs/architecture/repository-structure.md

Lead: Makar, frontend parts with Omar. One repository for code, tests, docs and diagram sources, so the report, the contract and the code are reviewed together.

Now (Assignment 1)

```
campus-social-network/
├── README.md                setup/run instructions, POC simplifications
├── Report.pdf               the submitted report, built from docs/
├── .github/CODEOWNERS       who reviews which area (3.1)
├── .gitignore               keeps .env, venvs, caches out of Git
├── backend/                 Makar
│   ├── main.py              FastAPI app: POST/GET /posts, GET /health
│   ├── test_main.py         pytest suite (8 tests)
│   └── requirements.txt
├── frontend/                Omar
│   ├── index.html           form + feed
│   └── app.js                fetch calls matching api-contract.md Part A
├── docs/
│   ├── requirements/        Salama (1.1–1.3, 2.1), Aro (1.4–1.5)
│   ├── architecture/
│   │   ├── api-contract.md  2.4 (Makar)
│   │   ├── behaviour-and-design.md 2.3 (Omar, Makar)
│   │   ├── repository-structure.md 2.5 (this file)
│   │   ├── adr/              2.7, one ADR per member
│   │   └── diagrams/         2.2, 2.3, 2.6 – Mermaid sources (editable, render on GitHub)
│   ├── project-management/  Section 3 (Aro)
│   ├── poc.md, demo-script.md Section 4 (Omar, Makar)
│   └── build_report.py       assembles Report.pdf (Aro)
```

Assignment 2 target

```
campus-social-network/
├── backend/
│   ├── app/
│   │   ├── main.py          app factory, routers mounted
│   │   ├── auth.py          Authcomponent
│   │   ├── policy.py         Access Policy (ADR-01) – one file, heavily tested
│   │   └── routers/          posts.py, groups.py, drafts.py, moderation.py, summaries.py,
├── ws.py
│   ├── services/            drafts.py, moderation.py, summary.py, llm_adapter.py
│   ├── models/              SQLModel tables matching er-diagram.md
│   ├── repositories/        queries and transactions
│   │   └── settings.py       reads configuration from environment variables
│   ├── alembic/              database migrations (ADR-03)
│   ├── tests/
│   │   ├── unit/            policy, services (LLM and DB mocked)
│   │   ├── api/             endpoint tests incl. role × endpoint authorization matrix
│   │   └── contract/         responses validated against openapi.json
│   ├── .env.example          variable names only, no values
│   └── pyproject.toml
├── frontend/
│   ├── src/
│   └── components/           Feed, PostEditor (rich text), DraftEditor, ModerationQueue,
```

SummaryPanel

			api/	typed client generated from openapi.json
			realtime/	WebSocket client, reconnect + refetch
			pages/	
			tests/	component tests (Vitest) + Playwright end-to-end
			.env.example	only public values (API base URL)
			package.json	
			shared/	
			openapi.json	exported from FastAPI; the single shared contract
			docker-compose.yml	api + postgres + mock identity provider for local dev
			.github/workflows/ci.yml	tests, lint, secret scan on every pull request
			docs/	same layout as now

How the structure reflects the architecture and the team

- Top-level folders match the C4 containers: `frontend/` = Web App, `backend/` = Backend API (+ Realtime hub), the database is defined by `backend/app/models` and `alembic/`.
- Inside `backend/app/`, files match the components in `diagrams/c4-component.md`, so a reviewer can go from diagram box to file directly.
- The only shared code is the contract: FastAPI exports `openapi.json`, the frontend's API client is generated from it. A contract change shows up as a diff in one file that both Makar and Omar review.
- `docs/` follows the report's numbering. `.github/CODEOWNERS` maps each folder to its lead, so GitHub requests the right reviewer automatically (Section 3.1).

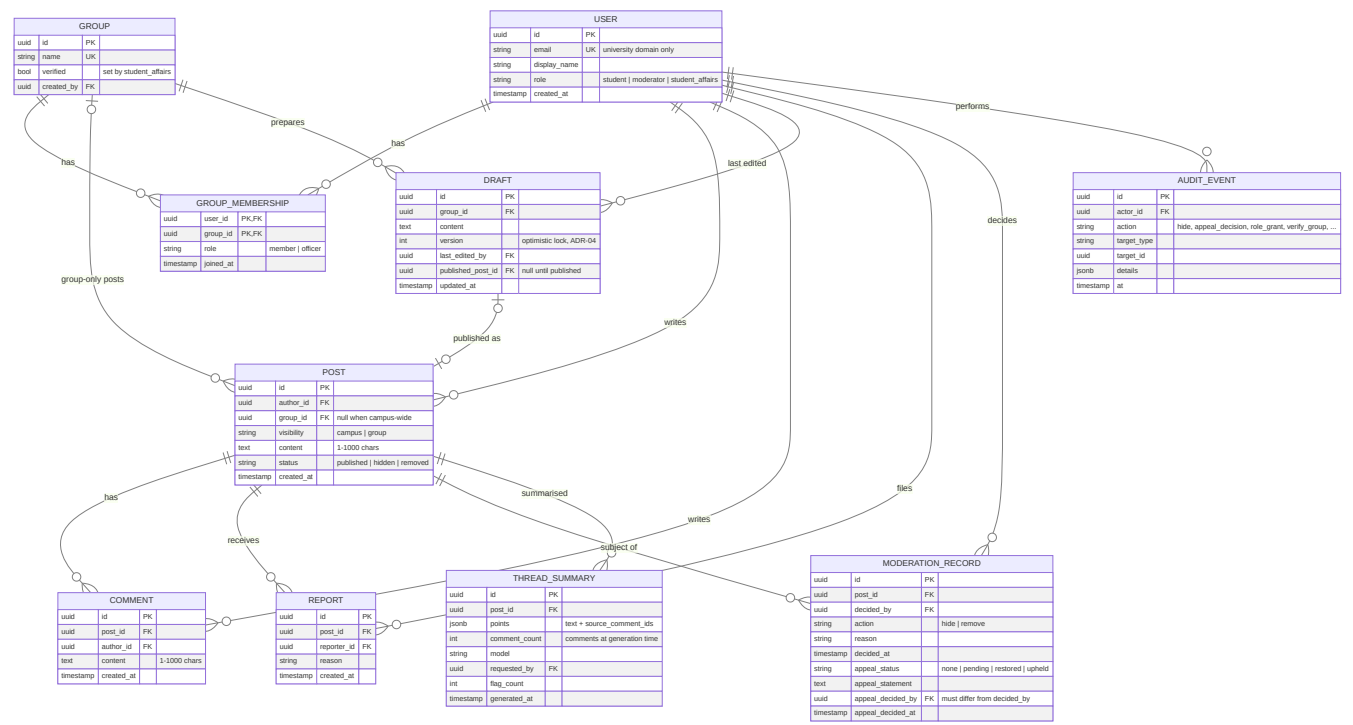
Keeping secrets out of the repository and browser code

- Secrets (database password, LLM API key, OIDC client secret, session signing key) are read from environment variables in `settings.py`. Locally they live in `backend/.env`, which is in `.gitignore`; only `.env.example` with variable names is committed.
- The browser never receives a secret: LLM and identity-provider calls are made by the backend. The frontend `.env` holds only public values (e.g. API base URL), because anything bundled into JavaScript is readable by users.
- The session is an `HttpOnly` cookie, so page scripts cannot read it.
- CI runs a secret scanner (gitleaks) on every pull request; a leaked key is rotated, not just deleted from history.
- The Assignment 1 POC needs no secrets at all.

2.6 Data Model

Source: docs/architecture/diagrams/er-diagram.md

Lead: Makar. Assignment 2 target model for PostgreSQL (ADR-03). The Assignment 1 POC stores only an in-memory post (id, author, content, created_at, see api-contract.md).



Entities

Entity	Purpose	Requirements
USER	A verified university account; <code>role</code> holds global powers	FR-11, NFR-7
GROUP	A club/community; <code>verified</code> drives the badge	FR-9, FR-12
GROUP_MEMBERSHIP	Who belongs to which group and who is an officer there	FR-4, FR-6, FR-12
POST	Published content with its audience and moderation status	FR-1, FR-2, FR-4
COMMENT	Discussion under a post	FR-3
DRAFT	Shared announcement being prepared by a group's officers	FR-7, NFR-8
REPORT	A user's complaint about a post	FR-10
MODERATION_RECORD	The decision on a post and its single appeal	FR-5, FR-6, FR-14, NFR-5
THREAD_SUMMARY	Stored AI summary with its sources, so it can be checked and flagged	FR-8, FR-15
AUDIT_EVENT	Append-only history of privileged actions	NFR-12

Relationships

- A user belongs to many groups and a group has many users, through `GROUP_MEMBERSHIP` (many-to-many). Officer is a role *on the membership*, so a person can be an officer of one group and a plain member of another.
- A post belongs to one author and, if group-only, one group. A post has many comments, reports, moderation records (one per decision) and summaries (regenerated as the thread grows).
- A draft belongs to one group and becomes at most one post when published.

Constraints (enforced in the database, not only in code)

Constraint	Supports
USER.email unique; domain checked at sign-in	FR-11, S5
POST.visibility = 'group' ⇔ group_id IS NOT NULL (CHECK)	FR-4, ADR-01
char_length(content) BETWEEN 1 AND 1000 on POST and COMMENT (CHECK)	FR-1, FR-3
UNIQUE (post_id, reporter_id) on REPORT	FR-10 (no duplicate reports)
appeal_decided_by <> decided_by (CHECK)	FR-14, ADR-01 second moderator
DRAFT.version incremented only by UPDATE ... WHERE id = ? AND version = ?	FR-7, NFR-8, ADR-04
Post status change and its MODERATION_RECORD written in one transaction	NFR-5
AUDIT_EVENT has no update/delete path in the API; DB role for the app has INSERT, SELECT only	NFR-12
Foreign keys ON DELETE RESTRICT for moderation/audit rows	NFR-5, NFR-12
Indexes on POST(created_at), POST(group_id, created_at), COMMENT(post_id, created_at)	NFR-1

How the model supports the policies

- **Audience control:** the feed query joins POST with the caller's GROUP_MEMBERSHIP ; group-only posts without a matching membership row are never selected.
- **Moderation and appeals:** hiding a post sets POST.status = 'hidden' but keeps the row, so content and reports remain available for the appeal (US-3, US-10).
- **AI checking:** THREAD_SUMMARY.points keeps the comment ids behind each point; flag_count records user reports of wrong summaries (FR-15).

ADR-01: Audience Visibility and Access-Control Policy

Source: docs/architecture/adr/adr-01-visibility-access-control.md

- **Status:** Accepted
- **Owner:** Salama
- **Date:** 2026-10-05 (first draft 2026-09-29)
- **Feasibility review:** Makar (backend), Omar (frontend) — buildable in FastAPI as one policy module; no objections.
- **Related:** FR-4, FR-5, FR-6, FR-9, FR-12, FR-14, NFR-7, NFR-10, NFR-12, US-3, US-4, US-6, US-7, US-10

Context

Students want posts to reach either the whole campus or only their group. Moderators need to remove harmful content, group officers want to manage their own group, and authors need a fair way to contest a decision. The rules must hold everywhere content leaves the server — REST responses, WebSocket events and text sent to the LLM — otherwise group-only posts leak.

Options considered

Visibility levels

1. Campus-wide only — simplest, but groups cannot share internal announcements.
2. **Campus-wide or group-only per post** (chosen) — covers the customer request with two rules.
3. Fine-grained lists (named users, friends-of-friends) — expensive to check on every request and every live event, and not requested by any stakeholder.

Who can moderate

1. Moderators only — consistent, but slow for large groups.
2. **Moderators anywhere + officers inside their own group** (chosen) — faster response, small blast radius if an officer account is misused.
3. Any group member can remove posts — easy to abuse (S5).

Appeals

1. No appeals — no recourse for a wrongly hidden post; hurts S1's trust.
2. **One appeal, decided by a different moderator** (chosen) — second pair of eyes, cheap enough for pilot scale.
3. Committee/admin panel — too heavy for a pilot.

Decision

- Every post has `visibility = campus` or `group`. A `group` post is visible only to members of its group (`GROUP_MEMBERSHIP`). Default is `campus`.
- All checks happen in one backend component (**Access Policy**, see `diagrams/c4-component.md`) called by the REST routers, the WebSocket hub before each broadcast, and the Summary service before building an LLM prompt. The frontend only hides buttons; it never decides access.
- Roles: `student` (default), `moderator`, `student_affairs` (global, on `USER.role`); `officer` (per group, on `GROUP_MEMBERSHIP.role`).
- Moderators can hide any post. Officers can remove posts in their own group only. Every hide/remove writes a `MODERATION_RECORD` in the same transaction as the post update.
- The author may appeal once. The appeal is assigned to a moderator other than the one who decided; the outcome (`restored` / `upheld`) is stored on the same record.
- Only `student_affairs` can verify a group (FR-9). Role grants are recorded in the audit log.

Consequences

- **Positive:** One place to test authorization (NFR-7, NFR-10): a role × endpoint test matrix covers REST and WebSocket paths.
- **Positive:** The ER model carries the policy directly (`visibility` , membership role, appeal fields on `MODERATION_RECORD`), so the database can enforce part of it with constraints.
- **Negative:** Every live event must be filtered per recipient, which costs CPU on the hub; acceptable at pilot scale (NFR-2), revisit if broadcasts become slow.
- **Negative:** Appeals assume at least two active moderators (risk R6); fallback is review by Student Affairs.
- **Open:** How the first `student_affairs` account is created — planned as a seed/admin script, not an API endpoint (open question Q-2).

ADR-02: AI Feature, Data Used, and How Users Check Its Output

Source: docs/architecture/adr/adr-02-ai-feature.md

- **Status:** Accepted
- **Owner:** Aro
- **Date:** 2026-10-05 (topic agreed 2026-09-29)
- **Feasibility review:** Makar (adapter + mock in FastAPI, citation check), Omar (summary panel with links to comments) — both feasible in Assignment 2.
- **Related:** FR-8, FR-15, NFR-10, NFR-12, US-9, A-5, A-6

Context

Assignment 2 requires an LLM integration that meets a real community need. Popular posts (events, course questions, housing) collect dozens of comments; students who arrive late must scroll through everything or miss the outcome. An LLM can be wrong, so its output must be checkable, and it must not become a way to read content the user is not allowed to see (ADR-01) or to send unnecessary personal data to a third party (NFR-10).

Options considered

1. **Thread summarisation on request, with citations** — clear need for S1; summarising visible text is low-risk; each point can be checked against the comments it cites.
2. **Automatic toxicity flagging for moderators** — useful to S3, but false positives directly affect what students may say and it overlaps with human moderation policy (ADR-01); errors are harder for users to check.
3. **AI drafting of announcements for officers** — helpful, but generated text is published under a group's name; verifying it is the officer's job, not the system's.
4. **No AI feature** — not allowed by the brief.

For option 1 we also considered **automatic summaries on every thread** (rejected: cost and unrequested AI text everywhere) versus **on request** (chosen).

Decision

Option 1, on request. - **Trigger:** a "Summarise" button on threads with ≥ 20 visible comments (FR-8). - **Data sent:** for each comment the requester can see — comment id, author display name, text. No emails, user ids, group membership, other threads, or hidden/removed comments. The Access Policy selects the comments before the prompt is built. - **Model:** called through an `LLMAdapter` interface. A deterministic `MockSummarizer` is used in development and tests; the real provider is set by configuration (A-5), with the key kept on the server only. - **Output format:** 3–6 points, each with `source_comment_ids`. The Summary service drops any point with no citations or with ids not in the input. - **How users check it:** the panel is labelled "AI-generated — check the linked comments"; clicking a point highlights its source comments; "Flag as wrong" stores a flag (FR-15). - **Stored:** the summary, the comment count at generation time and flag count (`THREAD_SUMMARY`), so it can be reviewed and regenerated once 10+ new comments arrive. - **Failure:** 8 s timeout, one retry, then "summary unavailable"; the thread works normally.

Consequences

- **Positive:** The feature addresses a clear S1 need and its errors are visible and correctable by the user, not hidden.
- **Positive:** Citations give the team a measurable quality check in Assignment 2: share of points whose cited comments actually support them, plus flag rate.
- **Positive:** The adapter + mock means tests and the demo need no paid API calls.
- **Negative:** Citations do not prove a point is a fair summary — a point can cite real comments but still misstate them; flags and spot checks are the mitigation.
- **Negative:** Comment text leaves our system for the provider (assumption A-6); a self-hosted model can replace it behind the same adapter if the university requires.
- **Negative:** Summaries can go stale as the thread grows; the UI shows the comment count they were based on.

ADR-03: Data Storage Approach

Source: docs/architecture/adr/adr-03-data-storage.md

- **Status:** Accepted
- **Owner:** Makar
- **Date:** 2026-10-05 (first draft 2026-09-29)
- **Feasibility review:** Omar — fine for the frontend, no direct impact beyond the API contract.
- **Related:** NFR-2, NFR-5, NFR-9, NFR-12, ADR-01, ADR-04, diagrams/er-diagram.md

Context

The POC keeps posts in memory. Assignment 2 needs persistent users, groups, memberships, posts, comments, drafts, reports, moderation records, summaries and an audit log. The data is strongly linked (post → group → membership → user; moderation record → post → moderator), and two rules are critical: a moderation decision must never be half-written (NFR-5), and two officers saving the same draft must not overwrite each other (NFR-8).

Options considered

1. **Document store (MongoDB)** — flexible schema and quick start, but our data is relational; joins, uniqueness ("one report per user per post") and cross-document consistency would have to be rebuilt in application code.
2. **SQLite** — zero setup and fine for the POC, but a single writer lock and no managed backups make concurrent drafts (ADR-04) and the recovery target (NFR-9) hard.
3. **PostgreSQL** — foreign keys, CHECK and UNIQUE constraints express the policies in er-diagram.md ; transactions make "hide post + write record" atomic; managed hosting gives point-in-time recovery; well supported by FastAPI through SQLAlchemy and Alembic.

Decision

PostgreSQL 16, accessed from FastAPI through SQLAlchemy with the async asyncpg driver, and Alembic for migrations. Local development runs Postgres in Docker Compose. Not used in the Assignment 1 POC, which deliberately stays in memory (see docs/poc.md).

Consequences

- **Positive:** Policy rules become database constraints (visibility ⇌ group, second moderator ≠ first, unique report), so a bug in one endpoint cannot break them (NFR-5).
- **Positive:** Optimistic locking for drafts is one conditional UPDATE ... WHERE version = ? (ADR-04) — no extra infrastructure.
- **Positive:** Point-in-time recovery on the managed service meets RPO 15 min (NFR-9).
- **Negative:** Every schema change needs a migration and review; slower than a schemaless store.
- **Negative:** Team must run Docker for local development; mitigated by one docker-compose up command in the A2 README.
- **Follow-up:** If the live feed (NFR-1) gets slow at pilot scale, add read indexes or a cache before considering a different store.

ADR-04: Collaboration and Concurrent Editing Approach

Source: docs/architecture/adr/adr-04-collaboration-concurrent-editing.md

- **Status:** Accepted
- **Owner:** Omar
- **Date:** 2026-10-05 (first draft 2026-09-29)
- **Feasibility review:** Makar — conditional UPDATE on a version column is straightforward in PostgreSQL/SQLModel (ADR-03).
- **Related:** FR-7, FR-13, NFR-4, NFR-8, US-8, diagrams/sequence-draft-handoff.md

Context

Group officers prepare announcements together and hand drafts to each other before publishing (S2). Two officers may edit the same draft at the same time. Assignment 2 must use WebSockets and a React rich-text editor, so the decision should not block live co-editing later, but it must be buildable and testable by a team of four within Assignment 2.

Options considered

1. **Pessimistic locking** — one officer checks out a draft, the others are read-only. Simple, but a forgotten checkout blocks the group and there is no presence information.
2. **Real-time co-editing with a CRDT/OT library (e.g. Yjs)** — best experience, but adds a sync server, binary document state and hard-to-test merge behaviour; overkill for handing off announcements.
3. **Optimistic concurrency + WebSocket presence** — each draft has a `version`. A save must name the version it was based on; if the draft moved on, the save gets 409 with the latest text. A WebSocket channel shows who is viewing and announces each new version.

Decision

Option 3. Saves go through REST (`PUT /drafts/{id}` with `version`), checked by a conditional `UPDATE ... WHERE version = ?`. The Realtime hub sends `draft.presence` and `draft.updated` events to officers of that group only. Not part of the Assignment 1 POC.

Consequences

- **Positive:** No edit is silently overwritten (NFR-8); the rule is one SQL statement and easy to test with two clients.
- **Positive:** Presence and update events make conflicts rare and visible.
- **Positive:** The WebSocket channel is reused for the live feed and moderation queue (FR-13).
- **Negative:** On conflict the officer merges by hand; two people cannot type in the same paragraph at once.
- **Negative:** The rich-text editor must show a side-by-side merge view — extra frontend work.
- **Later:** If simultaneous typing becomes a real need, Option 2 can replace the draft editor only; the REST contract for publishing stays the same.

3 Project Management and Team Collaboration

Source: docs/project-management/README.md

Lead: Aro, with input from all members.

3.1 Team structure

Member	Responsibilities	Code/docs they maintain	Backup reviewer
Salama	Product scope, requirements (1.1–1.3), architectural drivers (2.1), ADR-01; in A2: acceptance of each feature against its FRs, access-policy rules	docs/requirements/ (except stories), adr-01	Aro
Aro	User stories, validation & traceability (1.4–1.5), project plan (3), ADR-02, report assembly; in A2: test plan, AI summary feature UX, release checklist	docs/requirements/user-stories.md , requirements-review.md , docs/project-management/ , adr-02 , docs/build_report.py	Salama
Makar	POC backend, interfaces (2.4), repository structure (2.5), data model (2.6), component diagram, ADR-03; in A2: FastAPI services, database, migrations, Access Policy, WebSocket hub	backend/ , api-contract.md , er-diagram.md , c4-component.md , adr-03	Omar
Omar	POC frontend, C4 context/container (2.2), behaviour & sequence (2.3), README, demo, ADR-04; in A2: React app, rich-text and draft editor, realtime client, end-to-end tests	frontend/ , README.md , c4-context.md , c4-container.md , behaviour-and-design.md , sequence diagrams, adr-04	Makar

.github/CODEOWNERS encodes this table, so GitHub asks the right person to review.

Coordinating changes across areas. The API contract is the boundary between Makar's and Omar's work: any change to it is a pull request reviewed by both, and in A2 the frontend client is generated from openapi.json , so a mismatch breaks the build. A requirement change touches Salama's or Aro's files and must update the traceability matrix in the same PR.

Sharing knowledge. Every member must be able to defend the whole submission. Each reviewer of a PR must be able to explain it; a weekly 30-minute sync walks through what changed; before the oral exam each member presents a section they did not write and the others question them (Step 6 of the brief).

3.2 Development workflow

- **Assignment 1 (honest record):** small team, documents mostly, so commits went directly to master , each authored by the person responsible for that section. Reviews happened in person and in team chat.
- **Assignment 2 branches:** master is always runnable and protected. Work happens on short-lived branches feat/<area>-<topic> , fix/... , docs/... , merged within a few days.
- **Pull requests:** every change goes through a PR linked to an issue. At least one approval from the CODEOWNER of the touched area; changes to api-contract.md / openapi.json need both Makar and Omar. CI (tests, lint, secret scan) must pass. Squash-merge with an imperative message.
- **Code review checklist:** matches the contract? tests added? authorization handled in the Access Policy, not ad hoc? requirement IDs referenced? docs updated?
- **Issues and task tracking:** GitHub Issues, one per user story or technical task, labelled by area (backend , frontend , docs) and requirement ID (FR-7). A GitHub Projects board with columns Backlog → This week → In progress → Review → Done.
- **Recording decisions and requirement changes:** architecture decisions as new or superseding ADRs in docs/architecture/adr/ ; requirement changes as PRs that edit the requirement, keep its ID (or retire it, never reuse), and update the traceability matrix. The Git history is the change log.
- **AI development tools:** we used AI assistants (Claude Code) to draft parts of the documents, code and tests in Assignment 1, and will continue in Assignment 2. Rules: AI output is committed only by the member who owns that area and who has read and understood it; it goes through the same PR review and CI as any code; generated code

must come with tests that the member has run; no secrets or personal data are pasted into AI tools; anything a member cannot explain in the oral exam is rewritten or removed.

3.3 Development approach

Kanban with one-week iterations. We chose it over Scrum because we are four students with uneven weekly availability around other courses; fixed sprint commitments and ceremonies would cost more than they return. Kanban keeps work visible and limits work-in-progress (max 2 cards per person), while the weekly cycle gives us regular checkpoints.

- **Iterations:** each week starts with a 30-minute planning sync (pick cards for the week from the backlog) and ends with a demo of what works on `master`.
- **Prioritisation:** MoSCoW from `functional-requirements.md` — all *Must* stories before *Should*; within that, the architectural drivers (2.1) first, because they are the hardest to change later (Access Policy, data model, draft concurrency).
- **Feedback:** weekly demo inside the team; Salama and Aro check finished stories against acceptance criteria; feedback from the teaching team is turned into issues; a hallway test with 5 students for NFR-11 in week 5.
- **Testing, documentation and technical tasks:** a story is *Done* only when its tests pass in CI and its docs/contract are updated — testing and docs are part of each card, not a phase at the end. Technical tasks (CI, Docker, migrations) are cards on the same board, scheduled in week 1 so feature work is not blocked.

3.4 Risks

ID	Risk	Likelihood	Impact	Reduce it	If it happens
R1	Docs, contract and code drift apart	Medium	High	Contract-first; <code>openapi.json</code> generated client; PR checklist; traceability updated in the same PR	Week-6 consistency review fixes the report, not the code, where the code is right
R2	Group-only content leaks through one endpoint or live event	Medium	High	One Access Policy used by REST, WebSocket and Summary paths; role × endpoint test matrix	Hotfix, add the missing test case, audit logs for affected posts
R3	LLM gives wrong or misleading summaries	High	Medium	Citations required, uncited points dropped, label + flag button (ADR-02)	Raise threshold, hide feature behind a flag, keep the rest of the app working
R4	LLM provider unavailable, rate-limited or not free	Medium	Medium	Adapter + <code>MockSummarizer</code> ; demo can run on the mock	Switch provider or demo with the mock and explain
R5	WebSocket and draft-conflict logic takes longer than planned	Medium	High	Start in week 3 with presence + 409 only (no CRDT, ADR-04); two-client tests early	Ship conflict detection without presence; presence is a <i>Should</i>
R6	Fewer than two moderators available, so appeals stall	Low	Medium	Assumption A-3 recorded	Appeals escalate to Student Affairs
R7	A member is unavailable for a week (exams, illness)	Medium	Medium	Backup reviewer per area (3.1); small cards; knowledge-sharing sync	Backup picks up the area; scope cut by MoSCoW <i>Should</i> items
R8	A member cannot explain a part they did not write in the oral exam	Medium	High	Cross-presentation practice before the exam; AI-tool rule in 3.2	Extra practice session on weak sections
R9	Secret (LLM key, DB password) committed to Git	Low	High	<code>.env</code> ignored, <code>.env.example</code> only, gitleaks in CI	Rotate the key immediately, then clean history

3.5 Assignment 2 timeline

Week numbers count from the Assignment 2 release. Each milestone is shown in the weekly demo and checked by the listed test.

Week	Milestone — what will work	How we verify it
1	Docker Compose starts API + Postgres + mock sign-in; a user signs in with a university email, a non-university email is rejected; CI runs on every PR	<code>docker compose up</code> , sign-in demo, auth tests green
2	Users create and list posts stored in Postgres; join/leave groups; group-only posts hidden from non-members by REST	Role × endpoint tests for FR-1, FR-2, FR-4, FR-12
3	React app with rich-text post editor and comments; feed and comments update live in a second browser without reload	Playwright test with two browser contexts (FR-3, FR-13, NFR-4)
4	Two officers edit one draft: presence list visible, stale save gets 409 with merge view, publish creates one post	Two-client conflict test (FR-7, NFR-8)
5	Report → live moderation queue → hide → appeal decided by a second moderator; audit rows written. Usability hallway test (5 students)	Transaction + CHECK tests (FR-5, FR-6, FR-10, FR-14, NFR-5, NFR-12); NFR-11 result
6	Thread summaries with clickable citations and flagging (mock and real provider); load test and restore drill done; report and demo final	Citation tests (FR-8, FR-15); Locust report (NFR-1); restore log (NFR-9)

4 Proof of Concept

Source: docs/poc.md

Leads: Makar (backend), Omar (frontend, README, demo). Validated against US-1/US-2 acceptance criteria by Aro and against FR-1/FR-2 by Salama.

What it demonstrates

One complete interaction from the planned architecture — **creating a campus post**: browser form → `POST /posts` → backend validation → 201 response → post shown at the top of the feed. On load the page calls `GET /posts` to show existing posts newest first. Invalid input returns 422 and the page shows a readable error.

Part	File	Owner
Backend (FastAPI)	<code>backend/main.py</code>	Makar
Backend tests (8)	<code>backend/test_main.py</code>	Makar
Frontend (HTML + JS)	<code>frontend/index.html</code> , <code>frontend/app.js</code>	Omar
Contract	<code>docs/architecture/api-contract.md</code> Part A	Makar & Omar
Run instructions	<code>README.md</code>	Omar
Demo plan	<code>docs/demo-script.md</code>	Omar

Request and response match the contract

- Request `{author, content}` and the 201 body `{id, author, content, created_at}` are the Pydantic models in `main.py`; FastAPI serves the same schema at `/docs`.
- The 422 body is FastAPI's standard shape, as documented; `app.js` reads `detail[].loc` and `detail[].msg` from exactly that shape.
- Tests assert status codes, fields, ordering and error shapes.

Acceptance check (Aro, Salama)

Criterion	Result
US-1: valid post appears at the top of the feed without reload	Pass (browser test)
US-1: empty content → validation error, nothing created	Pass (<code>test_create_post_missing_content_returns_422</code> , <code>test_whitespace_only_content_returns_422</code> , browser)
US-2: posts newest first	Pass (<code>test_list_posts_newest_first</code>)
US-2: empty feed shows no error	Pass (<code>test_list_posts_empty</code> , browser)
FR-1 length rule 1–1000	Pass (<code>test_content_over_1000_chars_returns_422</code>)

What we simplified from the planned architecture

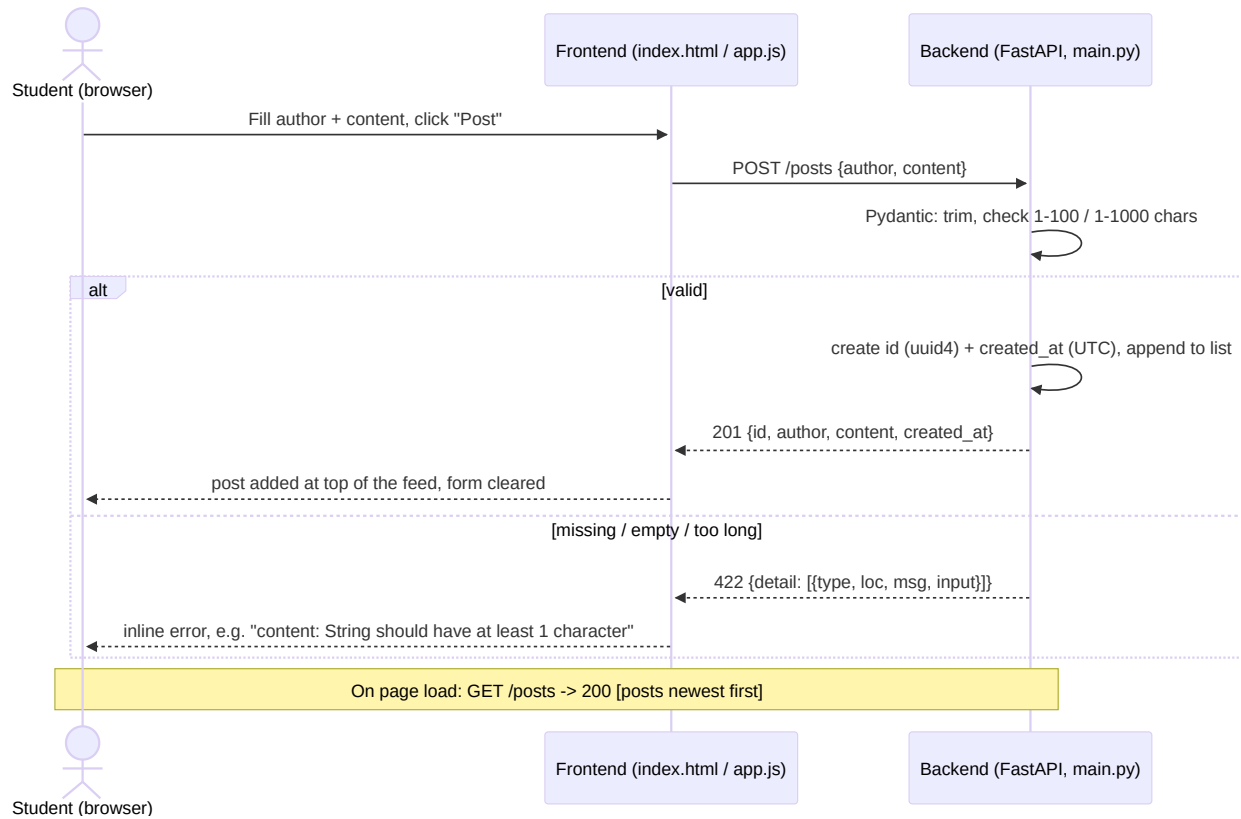
Planned (Assignment 2)	In the POC	Why it is acceptable here
React + rich-text editor	Plain HTML + JavaScript	Brief says React is not needed; same request/response either way
University sign-in, roles	<code>author</code> is free text	Shows the flow without an identity provider
PostgreSQL (ADR-03)	In-memory Python list, cleared on restart	No setup for the demo; storage is behind the endpoint, so it can be swapped
Access Policy, groups, visibility	Every post is public	Single interaction only

Planned (Assignment 2)	In the POC	Why it is acceptable here
WebSocket live updates	Page shows its own new post; others appear on reload	Brief says live updates are not needed
LLM summaries	Not present	Brief says AI integration is not needed
CORS restricted to the web app origin	CORS *	Local demo on two ports
Uniform error envelope for non-422 errors	Only 422 can occur	No auth/conflict cases exist yet

Proof-of-Concept Sequence — Create a Post

Source: docs/architecture/diagrams/sequence-create-post.md

Lead: Omar. The single interaction implemented in Assignment 1 (Section 4) and shown in the demo. Matches `backend/main.py`, `frontend/app.js` and `api-contract.md` Part A.



Demo Script (≤ 3 minutes)

Source: docs/demo-script.md

Lead: Omar. Rehearse with a timer; any member should be able to run it.

Before the demo (not timed)

- 1. Terminal 1: `cd backend && source .venv/bin/activate && uvicorn main:app --port 8000`
- 2. Terminal 2: `cd frontend && python3 -m http.server 5500`
- 3. Browser tab 1: `http://localhost:5500` — tab 2: `http://localhost:8000/docs`
- 4. Restart the backend just before starting so the feed is empty.

Script

Time	Who speaks	Action	What it shows
0:00–0:20	Salama	One sentence on the problem and product (<code>scope.md</code>)	Why the product exists
0:20–0:35	Omar	Show the empty feed	<code>GET /posts</code> → <code>[]</code> , US-2 "empty feed, no error"
0:35–1:05	Omar	Post as "Salama": "Study group for AI1220 at 6pm"	<code>POST /posts</code> → 201 → shown at the top: form → backend validation → response → display
1:05–1:20	Omar	Post a second one, show newest-first order	US-2 ordering
1:20–1:45	Makar	Submit spaces only as content	422 → "content: String should have at least 1 character"; nothing stored
1:45–2:10	Makar	Tab 2: <code>/docs</code> , open <code>POST /posts</code> schema	Running contract = <code>api-contract.md</code> Part A
2:10–2:40	Aro	What is simplified and why (<code>docs/poc.md</code> table)	Explains POC vs. architecture
2:40–3:00	Aro	One sentence on Assignment 2: React, WebSockets, Postgres, cited AI summaries	Links to ADRs

If the live demo fails

- Run `cd backend && pytest -q` (8 passed) as evidence the contract holds.
- Keep screenshots of the working flow on the laptop as a last resort.